

Identity Management in Hyperledger Sawtooth Transaction Families

This article follows several earlier ones on Hyperledger Sawtooth. It explores the concept of identity and its various ramifications in Hyperledger Sawtooth. It is targeted at open source technology enthusiasts with an interest in blockchain technologies and those who have a working knowledge of the Hyperledger Sawtooth project.

On-chain permissioning is required for transactor key and validator key permissioning. This feature requires that authorised participants for each role be stored. Currently, the only form of identification on the network is the entity's public key. Since lists of public keys are difficult to manage, an identity namespace is used to streamline the management of identities. The identity system in Hyperledger Sawtooth is an extensible role and policy based system for defining permissions in a way that can be used by other pieces of the architecture. This includes the existing permissioning components for the transactor key and validator key, but in the future, may also be used by transaction family implementations.

The identity namespace

The following list covers what the identity namespace does:

- Encompasses ways to identify participants based on public keys
 - Stores a set of 'permit and deny' rules called policies
 - Stores the roles that those policies apply to
- Policies describe a set of identities that have the permission to perform some action, without specifying the logic of the action itself. Roles simply reference a policy, providing a more flexible interface. For example, a policy could be referenced by multiple roles. So if the policy is updated, all roles referencing the policy are updated as well. An example of a role might be named 'transactor', and references the policy that controls who is allowed to submit batches and transactions on the network.

State

Policies

A policy has a name and a list of entries. Each policy entry has a list of type/key pairs. The type is either a *PERMIT_KEY* or a *DENY_KEY*. Each key in a type/key pair is a public key.

```
message PolicyList {
  repeated Policy policies = 1;
```

```
}

message Policy {

  enum EntryType {
    ENTRY_TYPE_UNSET = 0;
    PERMIT_KEY = 1;
    DENY_KEY = 2;
  }

  message Entry {
    // Whether this is a PERMIT_KEY or DENY_KEY entry
    EntryType type = 1;

    // This should be a public key or * to refer to all
    // participants.
    string key = 2;
  }

  // name of the policy, this should be unique.
  string name = 1;

  // list of Entries
  // The entries will be processed in order from first to
  // last.
  repeated Entry entries = 2;
}
```

Roles

A role is made up of a role name and the name of the policy to be enforced for that role. The data is stored in state at the address described above, using the following *protobuf* messages:

```
message RoleList {
  repeated Role roles = 1;
}
```